

Reconhecimento de Padrões Lógicos (XOR) com Redes Neurais MLP e Backpropagation

Trabalho complementar — extensão do estudo sobre Adaline aplicado a Redes Neurais Artificiais

Pedro Neto e Guilherme Kauã

Resumo

Este trabalho complementa o estudo anterior sobre o neurônio Adaline, abordando agora um problema que um único neurônio linear não é capaz de resolver: a função lógica OU EXCLUSIVO (XOR). Para isso, foi implementada do zero (sem uso de frameworks de aprendizado de máquina) uma rede Perceptron Multicamadas (MLP) com uma camada intermediária, treinada pelo algoritmo de Backpropagation. São apresentados a fundamentação teórica, a implementação em Python, e uma série de experimentos que avaliam a influência do número de neurônios na camada oculta, da taxa de aprendizagem e da inicialização aleatória dos pesos sobre a convergência da rede. Por fim, a rede é testada em outras portas lógicas (AND, OR, NAND, NOR), permitindo comparar a dificuldade de cada problema.

I. Introdução

O trabalho anterior apresentou o neurônio Adaline, um modelo de unidade única capaz de separar padrões linearmente separáveis por meio de um hiperplano de decisão. Entretanto, existem problemas — como a função lógica XOR — que não podem ser resolvidos por um único neurônio linear, pois suas classes não são linearmente separáveis no espaço de entrada.

Para superar essa limitação, surgem as redes Perceptron Multicamadas (MLP, do inglês Multi Layer Perceptron), que combinam múltiplas unidades de processamento organizadas em camadas, intercaladas por funções de ativação não-lineares. O treinamento dessas redes é realizado pelo algoritmo de Backpropagation (retropropagação do erro), uma generalização da regra delta utilizada no Adaline, que permite ajustar os pesos de todas as camadas a partir do erro observado na saída.

Este documento descreve a implementação de uma MLP com uma camada oculta para resolver o problema XOR, bem como um conjunto de experimentos que avaliam empiricamente o comportamento da rede sob diferentes configurações.

II. Fundamentação Teórica

A. O problema XOR

A função lógica XOR recebe duas entradas binárias x_1 e x_2 e retorna 1 quando exatamente uma das entradas é igual a 1, e 0 caso contrário. Diferentemente das funções AND e OR, não existe uma única reta capaz de separar as classes de saída 0 e 1 no plano cartesiano formado pelas combinações de entrada, conforme ilustrado na Figura 1.

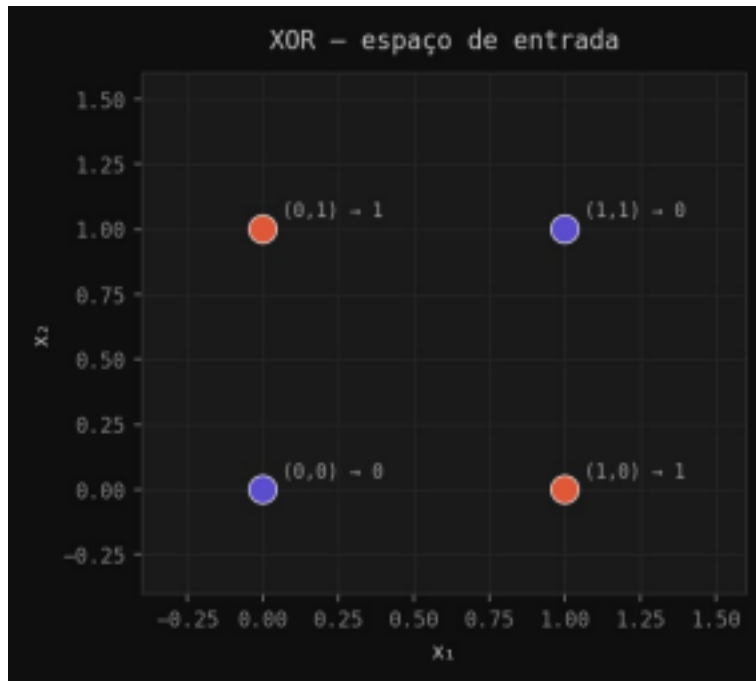


Fig. 1. Representação da função XOR no espaço de entrada. Não é possível traçar uma única reta que separe as classes 0 (roxo) e 1 (laranja).

B. Funções de ativação

Como a saída da rede precisa introduzir não-linearidade para resolver problemas como o XOR, foram implementadas três funções de ativação: a sigmoide binária (saídas entre 0 e 1), a sigmoide bipolar e a tangente hiperbólica (ambas com saídas entre -1 e 1). A Figura 2 mostra o comportamento de cada uma delas. Nos experimentos apresentados neste trabalho foi utilizada a função sigmoide bipolar em todas as camadas.

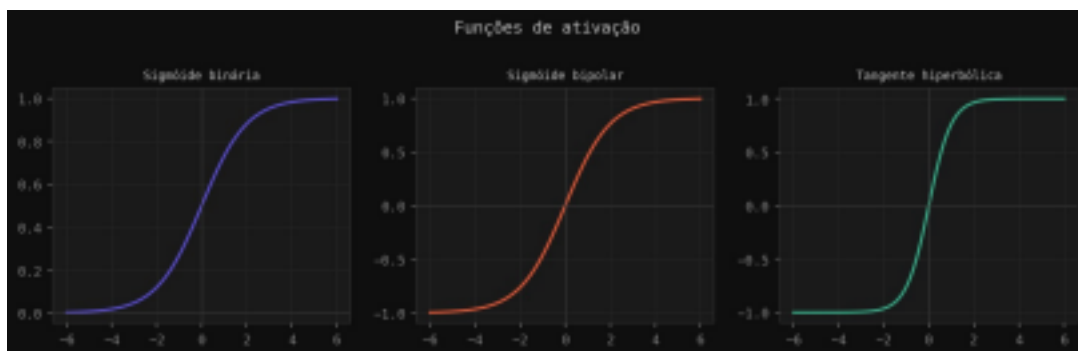


Fig. 2. Funções de ativação implementadas: sigmoide binária, sigmoide bipolar e tangente hiperbólica.

C. Arquitetura da rede e Backpropagation

A rede implementada possui uma camada de entrada com 2 neurônios (correspondentes às entradas x_1 e x_2), uma camada intermediária (oculta) com número variável de neurônios, e uma camada de saída com 1 neurônio. Todas as conexões possuem pesos e bias inicializados aleatoriamente no intervalo $[-0,5; 0,5]$.

No passo de propagação direta (forward), cada camada calcula a soma ponderada de suas entradas, seguida da aplicação da função de ativação. No passo de retropropagação (backward), o erro da camada de saída é calculado pela diferença entre a saída desejada e a saída obtida, ponderada pela derivada da função de ativação. Esse erro é então propagado para a camada oculta por meio dos pesos da camada de saída, permitindo calcular o ajuste de todos os pesos da rede, de forma análoga à regra delta do Adaline, porém aplicada camada a camada.

O critério de convergência adotado nos experimentos considera que a rede convergiu quando todas as saídas produzidas para os quatro padrões de entrada do XOR estão a uma distância menor que 0,1 dos valores desejados, com um limite máximo de 10.000 épocas de treinamento.

III. Materiais e Métodos

A rede foi implementada em Python, utilizando apenas a biblioteca NumPy para operações matriciais e Matplotlib para geração dos gráficos, sem o uso de frameworks de aprendizado profundo. O código está organizado em dois módulos principais: `mlp.py`, que contém a implementação da classe MLP (inicialização, forward, backward e treinamento), e `experimentos.py`, que reproduz o experimento principal do XOR e executa os experimentos complementares descritos a seguir.

Foram definidos quatro experimentos, além da reprodução do caso principal:

Experimento principal: rede com 2 entradas, 4 neurônios na camada oculta e 1 saída, taxa de aprendizagem (lr) igual a 0,2, ativação sigmoide bipolar e semente (seed) de inicialização igual a 42.

Experimento A: variação do número de neurônios da camada oculta (2, 3, 4 e 5), mantendo $lr = 0,5$ e $seed = 42$.

Experimento B: variação da taxa de aprendizagem (0,1 a 0,5, em passos de 0,1), mantendo 4 neurônios ocultos e $seed = 42$.

Experimento C: variação da semente de inicialização dos pesos (seeds de 0 a 4), mantendo 4 neurônios ocultos e $lr = 0,2$.

Experimento de funções lógicas: treinamento da mesma arquitetura (4 neurônios ocultos, $lr = 0,5$, $seed = 42$) para as portas AND, OR, NAND, NOR e XOR, permitindo comparar a dificuldade de cada problema.

IV. Resultados

A. Experimento principal

A rede com 4 neurônios na camada oculta, $lr = 0,2$ e $seed = 42$ convergiu em 1.132 épocas. As saídas finais da rede para os quatro padrões de entrada foram: $(0,0) \rightarrow 0,0123$; $(0,1) \rightarrow 0,9067$; $(1,0) \rightarrow 0,9002$; $(1,1) \rightarrow 0,0252$ — todas próximas dos valores desejados (0, 1, 1 e 0, respectivamente), confirmando que a rede aprendeu corretamente a função XOR. A Figura 3

mostra a evolução do erro absoluto total ao longo das épocas de treinamento.

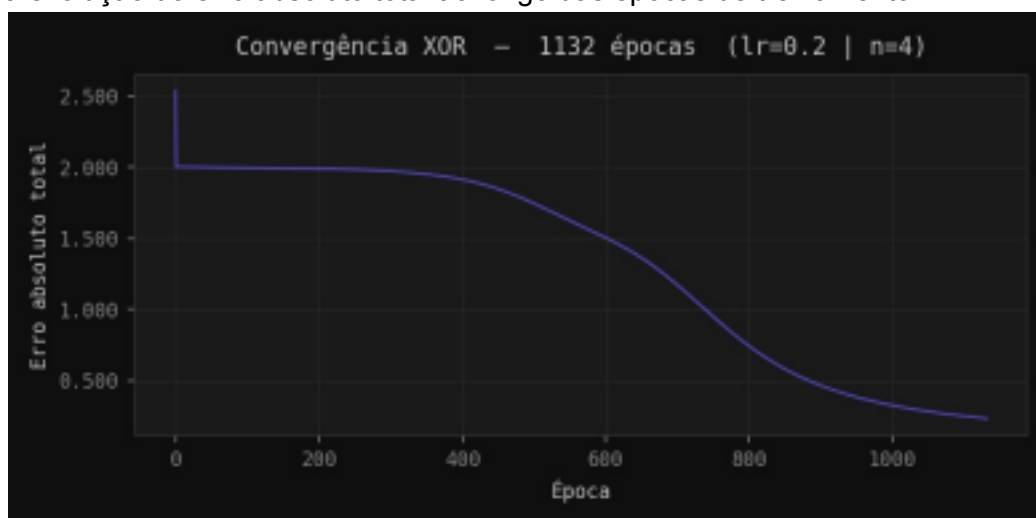


Fig. 3. Curva de convergência do experimento principal (4 neurônios ocultos, $lr = 0,2$, $seed = 42$): 1.132 épocas até o critério de parada.

B. Experimento A — número de neurônios na camada oculta

Com $lr = 0,5$ e $seed = 42$, redes com 2, 3, 4 e 5 neurônios na camada oculta convergiram em 543, 520, 465 e 446 épocas, respectivamente. Observa-se que o aumento do número de neurônios ocultos tende a reduzir o número de épocas necessárias para a convergência, embora o ganho seja decrescente — a diferença entre 4 e 5 neurônios é pequena, sugerindo que, para o problema XOR, poucos neurônios adicionais já trazem capacidade suficiente para representar a não-linearidade necessária.

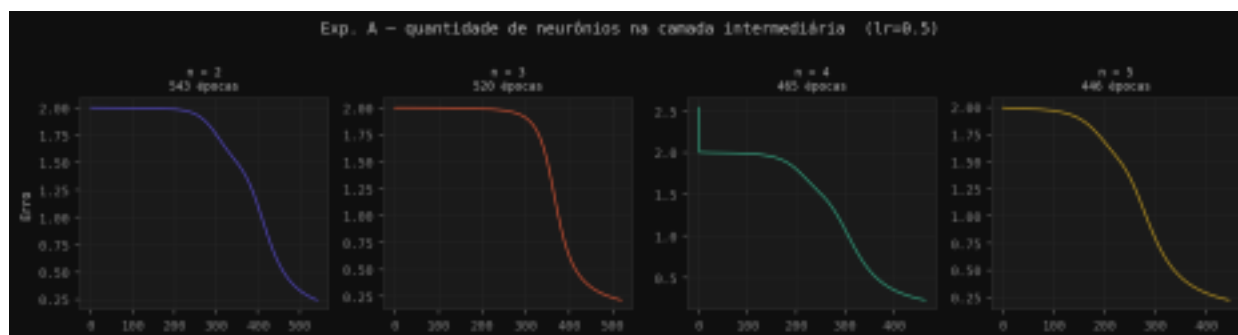


Fig. 4. Curvas de convergência para 2, 3, 4 e 5 neurônios na camada oculta ($lr = 0,5$, $seed = 42$).

C. Experimento B — taxa de aprendizagem

Mantendo 4 neurônios ocultos e $seed = 42$, foram testadas taxas de aprendizagem de 0,1 a 0,5. O número de épocas até a convergência foi de 2.250 ($lr = 0,1$), 1.132 ($lr = 0,2$), 760 ($lr = 0,3$), 575 ($lr = 0,4$) e 465 ($lr = 0,5$). Os resultados mostram uma relação claramente inversa entre a taxa de aprendizagem e o número de épocas necessárias: taxas maiores aceleram a convergência neste problema, sem causar instabilidade dentro da faixa testada. Em problemas mais complexos, no entanto, taxas muito altas podem levar a oscilações e impedir a convergência, de modo que esse parâmetro deve ser ajustado com cautela.

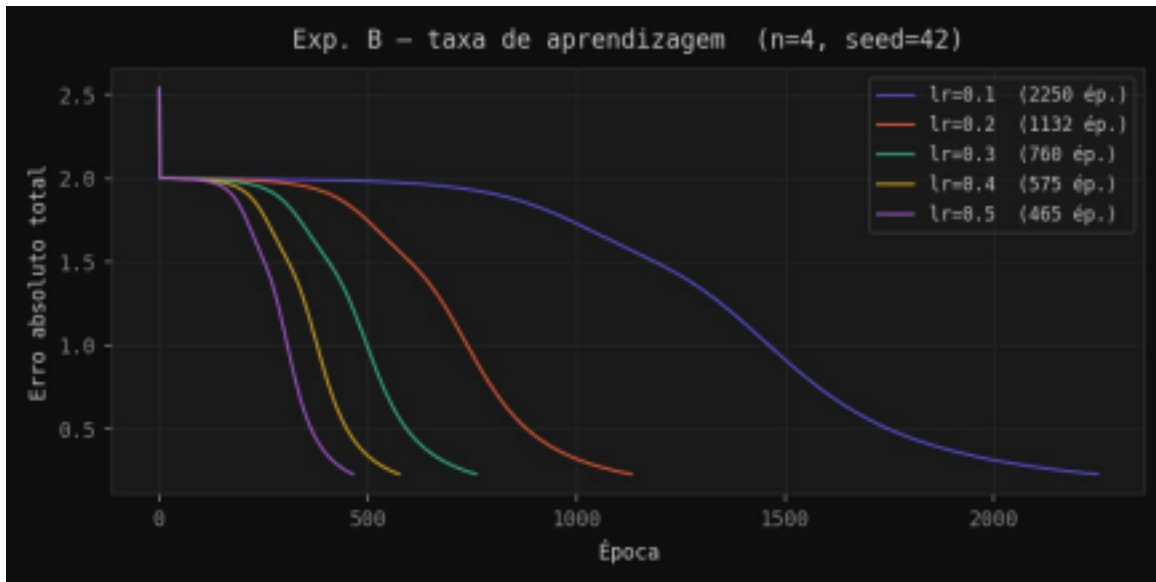


Fig. 5. Curvas de convergência para taxas de aprendizagem entre 0,1 e 0,5 (4 neurônios ocultos, seed = 42).

D. Experimento C — inicialização dos pesos

Com 4 neurônios ocultos e $lr = 0,2$, foram testadas cinco sementes de inicialização aleatória dos pesos (0 a 4), resultando em 1.112, 969, 1.484, 1.208 e 1.032 épocas até a convergência, respectivamente. A variação observada — de 969 a 1.484 épocas, uma diferença de mais de 50% entre o melhor e o pior caso — evidencia que a inicialização aleatória dos pesos tem influência significativa sobre o tempo de treinamento, ainda que todas as sementes testadas tenham levado a uma solução satisfatória para o problema.

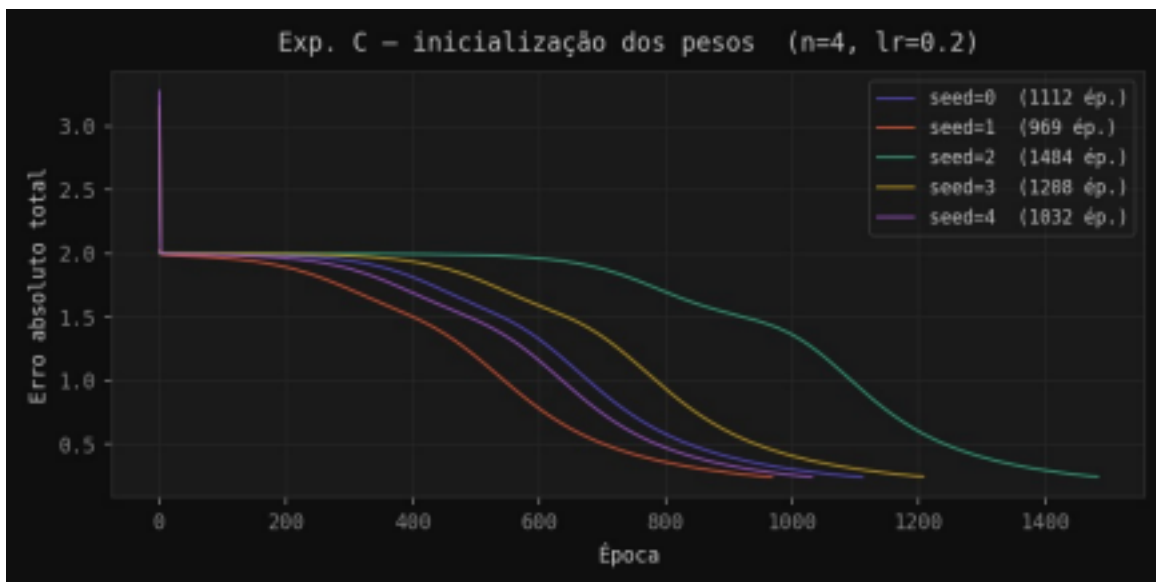


Fig. 6. Curvas de convergência para cinco inicializações aleatórias diferentes (4 neurônios ocultos, $lr = 0,2$).

E. Funções lógicas AND, OR, NAND, NOR e XOR

Utilizando a mesma arquitetura (4 neurônios ocultos, $lr = 0,5$, seed = 42), a rede foi treinada

para reproduzir as portas lógicas AND, OR, NAND, NOR e XOR. O número de épocas até a convergência foi de 622 (AND), 103 (OR), 130 (NAND), 319 (NOR) e 465 (XOR). Nota-se que as funções OR e NAND, cuja saída positiva está associada à maioria das combinações de entrada, convergem muito mais rapidamente. Já o AND e o XOR, em que a saída 1 ocorre para poucas combinações (ou depende de uma relação não-linear entre as entradas, no caso do XOR), exigem mais épocas de treinamento.

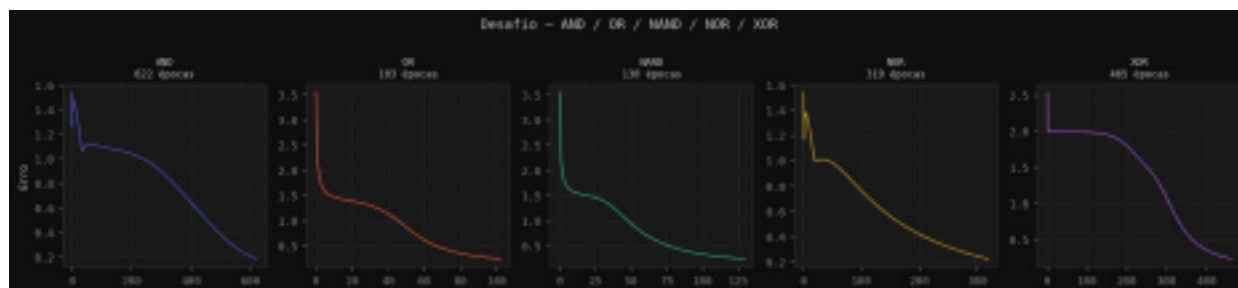


Fig. 7. Curvas de convergência da MLP para as portas lógicas AND, OR, NAND, NOR e XOR (4 neurônios ocultos, $lr = 0,5$, $seed = 42$).

V. Discussão

Os experimentos confirmam que a adição de uma camada oculta com função de ativação não linear é suficiente para que a rede aprenda a função XOR, problema que não é linearmente separável e, portanto, não pode ser resolvido por um único neurônio Adaline ou Perceptron. Além disso, os resultados mostram que parâmetros como o número de neurônios ocultos, a taxa de aprendizagem e a inicialização dos pesos influenciam diretamente a velocidade de convergência, ainda que, dentro das faixas testadas, não tenham impedido a rede de encontrar uma solução.

Assim como observado no trabalho com o Adaline, a presença de números aleatórios na inicialização dos pesos impossibilita a garantia de repetibilidade exata dos resultados, embora o uso de uma semente fixa ($seed$) permita reproduzir um experimento específico.

VI. Conclusão

Este trabalho complementou o estudo do neurônio Adaline ao abordar um problema não linearmente separável; a função XOR; por meio de uma rede Perceptron Multicamadas treinada com Backpropagation. A implementação, feita sem o uso de frameworks prontos, permitiu observar de forma direta o funcionamento dos passos de propagação direta e retropropagação do erro, bem como a influência de hiperparâmetros como o número de neurônios ocultos, a taxa de aprendizagem e a inicialização dos pesos sobre o desempenho da rede. Os experimentos com outras portas lógicas reforçam a ideia de que a dificuldade de aprendizado está relacionada à complexidade da fronteira de decisão de cada problema.

Material Complementar

O código-fonte completo da implementação (incluindo a classe MLP, os scripts de experimentos e as figuras geradas), bem como uma versão organizada dos arquivos para consulta, estão disponíveis nos links abaixo:

Repositório (código-fonte): <https://github.com/netod3v/mlp-xor-backpropagation>

Pasta com arquivos e resultados:

<https://drive.google.com/drive/folders/10NBpG56Jl10GrRszfZG1st-DtjdQFKQc?usp=sharing>

Observação: os links acima devem ser substituídos pelos endereços reais do repositório Git e da pasta do Google Drive antes do compartilhamento com o restante da equipe, com permissão de visualização habilitada para todos os integrantes.

Referências

[1] A. Braga, A. Carvalho, and T. Ludermir, Redes neurais artificiais: teoria e aplicações. LTC, 2000.

[2] L. Fausett, Fundamentals of neural networks: architectures, algorithms, and applications. Prentice-Hall Englewood Cliffs, NJ, 1994.

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back propagating errors", Nature, 1986.